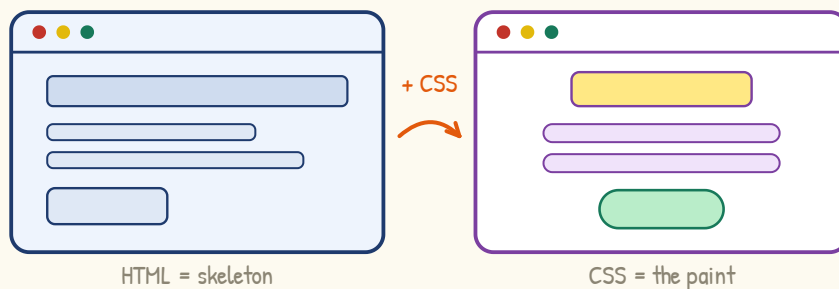


Chapter 1

HTML & CSS

Foundations

Structure the page with **HTML** · Style it with **CSS**



SOURCE : freeCodeCamp x Scrimba – “Become a Fullstack Developer from Scratch”

MODULE : HTML (00:01:27) + Intro to CSS (01:04:19)

COVERS : tags · nesting · attributes · document structure · lists · links · selectors · box model · centering · flexbox

GOAL : build & deploy a real page – Mars article, Google clone, personal site

⚡ TEACHING PHILOSOPHY

“First make it work, then make it better.” – build first, learn the formal theory after. Also: *the only way to learn is to type the code yourself* (500+ challenges).

1 What is HTML?

★★★★★

DEFINITION

HTML = HyperText Markup Language. It is the language used to give a web page its **structure & content** – headings, text, images, buttons, links.

Not a programming language – a markup language.

Text is made prominent by **wrapping it in HTML tags** – like choosing “Heading 1” in a word processor.

<h1>I code, therefore I am**</h1>**

▲
opening tag

▲
closing tag (has a / slash)

- Only text **between** the opening and closing tag gets formatted.
- Main file is normally named **index.html**. You can **drag & drop** any local **.html** file into a browser to render it.

► Core text tags

Tag	Meaning	Use
<h1>	Heading 1 – biggest	Main page / article title
<h2>	“smaller sibling” of h1	Section heading
<h3>	Heading 3	Sub-heading
<p>	P = paragraph	Normal body text

REMEMBER

Headings run **h1** → **h6**. Same syntax every time – **only the number changes**. Bigger number = smaller heading.

2 Images & attributes

★★★★★

▲ ▲
attribute + value attribute + value

SELF-CLOSING TAG

**** needs **no closing tag**. Some write **** to signal it closes itself – **optional, not required**.

- **src** = source → which image to render. Same folder ⇒ just the filename.
- Images can also be **borrowed from the web** – right-click an image online → Copy image address → paste into **src**.
- **.jpeg** and **.png** are both file formats, used the exact same way.

△ THE WIDTH “HACK”

An image only takes the width it needs, while a **<p>** takes the full width. **width="100%"** makes the image scale with the browser – but it is not valid HTML per the specification. It's taught as a deliberate hack; the **proper way is CSS**, learned later.

3 Nesting & the `div`

★★★★★

DEFINITION

Nesting = the act of placing HTML elements inside other HTML elements.

```
<div>
  <h1>...</h1>
  
  <h3>...</h3>
  <p>...</p>
</div>
```

div

Short for divider – though container describes it better.

It is *invisible* & changes no layout by itself, but lets you target a whole group at once with CSS or JavaScript.

Indent children for readability – the browser doesn't care, humans do.

↳ "A div inside a div inside a div – this is web development in a nutshell."

► The HTML document tree

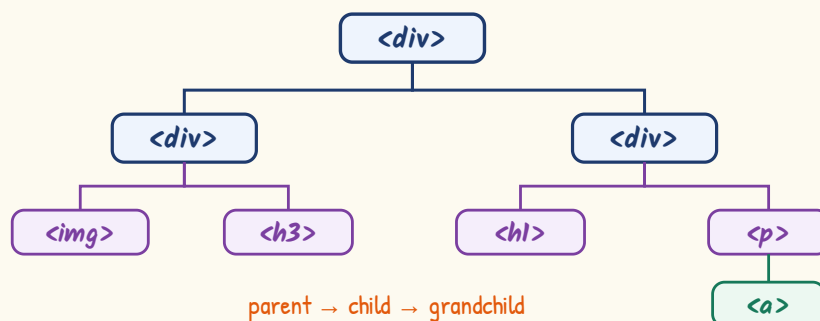


Fig 1 – nested elements form a family tree = the HTML document tree

IMPORTANT

Because tags nest inside tags, the document behaves like a tree – trunk → branches → smaller branches.
Terms used: *parent, child, children*.

► Dev Tools – inspect & edit any live site

1. Right-click on any element → Inspect (steps may differ slightly per browser).
2. The *developer tools* panel opens → *Elements* tab shows the page's HTML.
3. Double-click text to change it; change `src` to swap an image.

△ COMMON CONFUSION

Dev-tools edits are local only. Refresh the page and they are gone – the fresh data comes from the site's database. *You never change the real site.* (If an image won't swap, the site may use `srcset` – delete it too.)

4 Buttons & input fields

★★★★★

These are the elements that make a website **interactive**.

<button>Sign up</button>

- Text goes **between** the tags.
- Comes styled by default (background + border) & is clickable.
- Does nothing on its own – needs JavaScript to act.

<input type="text" placeholder="Enter username">

- Self-closing, like ****.
- **type** decides **what kind of input** it is.
- **placeholder** = hint text for the user.

► The **type** attribute – one tag, many tools

type value	What the browser renders
text	Plain text box – the most common
password	Characters are masked – real passwords are never exposed
date	Date picker (e.g. birth date at sign-up)
time	Time picker (e.g. when to send a notification)
color	Full colour picker – drag the dots for any colour
file	Upload button → choose file(s) from device storage

⚡ EXAM TIP

Being good at Googling is a huge part of being a good programmer – the two are hard to separate. Search pattern taught: **what it is + your intent** → “html/input upload image files” → MDN gives **type="file"**.

5 Anchor tags (links)

★★★★★

DEFINITION

The anchor tag **<a>** creates links between web pages or to external websites. **It is what makes the web a “web”.**

Check out my projects

<!-- external link, opens in a NEW TAB -->

here

- **href** = **hypertext reference** → decides **where the link points**. Fancy scary word – don't worry about it.
- Without **href** the anchor is not clickable – it just looks like a paragraph.
- **target="_blank"** = open in a **new tab**. A cryptic name – “new-tab = yes” would make more sense, but this is how it is.

INLINE LINKS

An **<a>** can be **nested inside a paragraph** so that only one word becomes clickable:

<p>We protect your privacy. Learn how</p>

6 Proper HTML document structure

★★★★★

"HTML is quite like us humans — it has a **head** and a **body**."

```
<!doctype html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
    <title>google.com</title>
  </head>
  <body>
    <!-- everything visible -->
  </body>
</html>
```

Part	Holds
<html>	Wraps all other tags
<head>	Metadata — not rendered
<body>	Everything visible on the page

The **<link>** to CSS goes in the **head** — it changes the layout but is not itself rendered.

What lives in the head?

- **<link>** to stylesheets.
- **<title>** → the text shown in the **browser tab**.
- Social-share metadata → lets a site show a **different heading & image** on a Twitter card than the article itself uses.

⚠ THE DOCTYPE

<!doctype html> is not an **HTML tag** — it is metadata telling the browser "this is **HTML5**, not **HTML4**". Omit it and the browser assumes an older version, which can cause **quirks**. In ~99% of cases there is no visible difference, but production sites must have it. Upper- or lowercase both work.

7 Lists

★★★★★

Two tags are always needed: **one wrapper + one tag per item**.

```
<ol> ← ordered
  <li>Norway</li>
  <li>Germany</li>
</ol>
```

Tag	Meaning
	Ordered list → numbers
	Unordered list → dots
	List item — goes inside

- The browser **auto-numbers** **** items and indents them from the left.
- Swap **ol** → **ul** and the numbers become bullets. Nothing else changes.

WHY LISTS MATTER

Lists look boring by default, but with **CSS** they can be styled freely. **Navigation bars** are usually a **** with each menu category as an **** — inspect **BBC Sport's navbar** to see it. **Lists are all over the web.**

8 Project - Google clone in 4 tags ★★★★★

Google is the **most-visited website in the world** – and its homepage needs only **four HTML elements** (the CSS is supplied separately).

```

<input type="text">
<button>Google Search</button>
<button>I'm Feeling Lucky</button>
```

9 File paths ★★★★★

Syntax	Starts looking from
<code>picture.jpg</code>	The same folder as the current page
<code>images/pic.jpg</code>	The images folder inside the current folder
<code>./images/pic.jpg</code>	Same as above – ./ explicitly means current directory
<code>../images/pic.jpg</code>	Go up to the parent directory first
<code>/images/pic.jpg</code>	Start from the project root directory

TIP

./ is **functionally identical** to writing nothing – it is a style preference that visually reminds you the file is local. Use **../** to climb two levels (e.g. from `src/css/` back to root), or just start with **/** – more concise.

10 Alt text - accessibility ★★★★★

```

```

Three reasons every `<img>` needs an `alt`:

1. **Accessibility** – a screen reader can read text, but needs `alt` to describe an image.
2. **Fallback** – if `src` is misspelled the image breaks, and the alt text explains what should be there.
3. **SEO** – helps you **rank higher on Google**; inaccessible sites may be punished with lower ranking.

△ RULES FOR GOOD ALT TEXT

- ✗ Don't start with "image of..." – the screen reader already knows it's an image.
- ✓ Keep it under **125 characters** – many screen readers cut off after that.
- ✓ Trick: describe it **as if over the phone** – convey the intent, not just the facts. "a boxing match" ✗ → "Muhammad Ali in the ring screaming at Sonny Liston after knocking him out" ✓

Deploying to the world – Netlify



Fig 2 – deploy flow: free, easy, used by professionals

PART B - CSS

11 What is CSS?

★★★★★

DEFINITION

CSS = Cascading Style Sheets – the language used for styling websites. It turns dull default HTML into a polished, designed product.

► Linking CSS to HTML (external CSS)

```
<link rel="stylesheet" href="styles.css">
```

▲
the relation

▲
where the sheet lives

Goes inside `<head>`. Tip: the instructor Googles "how to link to CSS" *almost every single time* – never be ashamed of it.

► The anatomy of a CSS rule

body { ----- curly brackets
selector
color : **red** ; ← semicolon is required!
property value

△ SYNTAX RULE

Property is followed by a **colon** : and the value by a **semicolon** ; – you must have that notation, otherwise you break the example.

► First properties met

Property	Does what
color	Colour of the text (not the background!)
background	Background colour of the element
font-size	Size of font, e.g. 25px
font-weight	Thickness – lighter / normal / bold
text-align	left / right / center
margin-top	Space above the element

px

Pixel = the **smallest unit** a screen can render – a tiny square. **400px** = 400 side by side.

HEX CODES

Another way to write colour: **#8B4513** = how much **Red**, **Green**, **Blue**. Names like **saddlebrown** also work.

⚡ DESIGN RULE OF THUMB

Never use pure black on pure white – unpleasant for the eyes; use a softer black. CSS comments: `/* ... */`.

12 Block vs inline elements

★★★★★



RULE OF THUMB

"Display block stacks on top."

A block element **blocks out all remaining horizontal space** in its container so nothing else can sit beside it – regardless of how wide its content actually is. The next element is therefore pushed underneath.

Default display	Elements
block	<code><div></code> , <code><p></code> , headings
inline	<code><button></code> , <code><a></code> , <code></code> *

* An image behaves inline by default, which is why the logo and input can end up side-by-side on a wide browser – fix with `display: block`.

13 Classes - the flexible selector

★★★★★

⚠ THE PROBLEM WITH ELEMENT SELECTORS

Writing `div { margin-top: 100px; }` selects every single div in the document – possibly thousands. Add a second div (e.g. a button wrapper) and the spacing breaks. **Too generic.**

HTML – add the attribute

```
<div class="main"> ... </div>
```

CSS – select with a dot

```
.main {
  margin-top: 100px;
}
```

- `class` is an **attribute built into HTML**; the name is yours to choose.
- Select it in CSS with **dot notation** → `.main`, `.btn`, `.card`.
- Naming convention shown: `logo-img`, `search-input` – **descriptive name + element**. Plenty of conventions exist; it does not matter which you use.

PRO PRACTICE

Don't put `margin-top` on the logo image itself – **wrap elements in a div and position them via the container**. If the logo is later swapped for a video or a doodle, the layout survives.

14 The Box Model

★★★★★

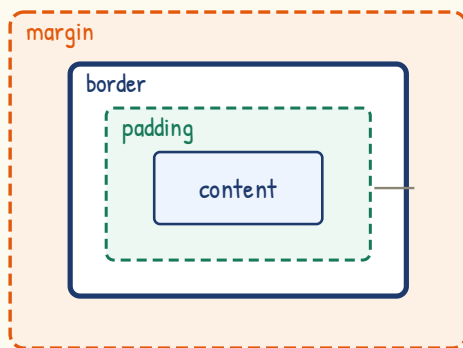


Fig 4 – content → padding → border → margin

NAME

All of this together is called the **BOX MODEL** – a name that “sounds more complex than it is”. It is just **spacing outside + border + spacing inside + the content**.

MARGIN

Space on the **OUTSIDE** of an element.

PADDING

Space on the **INSIDE** – between the content and the element's edge.

BORDER

Sits **between** padding and margin.

► Syntax – long form & shorthand

```
margin-top: 10px;
margin-bottom: 10px;
margin-left: 10px;
margin-right: 10px;
```

```
/* identical shorthand */
margin: 10px;
```

```
padding: 20px; /* all 4 sides */
```

Shorthand works for **padding** too. Default padding of a div = **0**.

► Border & border-radius

```
border: 2px solid red; /* thickness | style | colour */
border-radius: 30px; /* softens the corners */
```

- Styles include **solid**, **dotted**, **dashed** – but in 99.9% of cases you want **solid**.
- **border-radius** stops changing past a point – the corners have become **half-circles**.
- The **bigger the box, the more radius** is needed for the same soft look.

DESIGN TIPS

- ✓ **Reuse an existing colour** for the border instead of adding a new one – a common web-design trick.
- ✓ Buttons usually have **more padding left/right than top/bottom** (e.g. 10px vs 20px).

15 Margins in detail

★★★★★

⚠ BROWSER DEFAULT

The browser *automatically* applies `body { margin: 8px; }` so content doesn't touch the edges. Writing it yourself changes nothing – set `margin: 0` to remove it and see elements touch the browser edges.

⚠ COLLAPSING MARGINS – easy to forget!

Red box has `margin-bottom: 15px`, blue box below has `margin-top: 20px`. Expected gap = 35px. Actual gap = 20px.

Vertical margins *collapse into each other* – the larger value wins, they do not add up. The smaller one only "kicks in" once it becomes the larger.

There are scenarios where this does *not* happen (outside the scope of the lecture). Just be aware: if your margins don't seem to add up, it's probably collapsing margins.

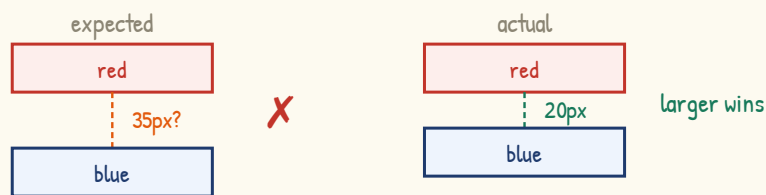


Fig 5 – collapsing margins: 15 + 20 ≠ 35

16 Centering with margins

★★★★★

A famously scary topic – the Stack Overflow question on horizontal centering has been viewed **4.5 million times**. If it confuses you, you are in very good company.

3 CONDITIONS – ALL REQUIRED

1. Element must be **display: block** → it must command the full horizontal space.
2. Element must have a **width** → it can't centre if it's wider than its container.
3. Set **margin-left: auto** and **margin-right: auto**.

THE CENTERING RECIPE

display: block; width: 200px; margin-left: auto; margin-right: auto;

WHY **auto** WORKS

auto means "take up all the available space". Set only `margin-left: auto` and the element is pushed fully **right**. Add `margin-right: auto` and both sides fight for the space, agree to split it evenly – and the element lands in the centre.

⚠ TRAP

`margin: auto` on a **button** *does nothing* – buttons are inline by default. Add `display: block` first. You needn't always set width manually – a button already has an implicit width.

17 Flexbox

★★★★★

DEFINITION

A flexbox layout **always consists of a container + its direct children**. Setting `display: flex` on the container forces the children onto **one row** – overriding their default display: block.

```
/* HTML */
<div class="nav-wrapper">
  <div>Shoes</div>
  <div>Hoodies</div>
</div>
```

```
/* CSS */
.nav-wrapper {
  display: flex;
  justify-content: center;
}
```

► justify-content – position along the main axis

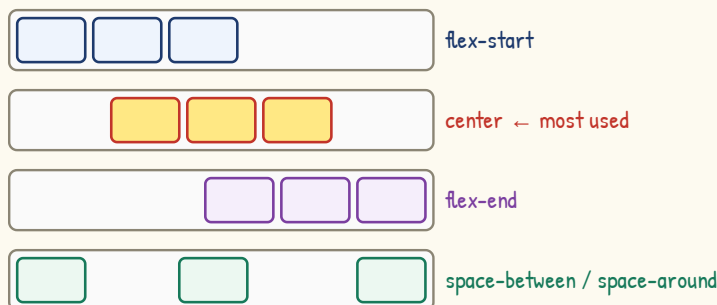


Fig 6 – the same three items under different justify-content values

- The layout flows **left → right along the horizontal axis**.
- `space-around` → white space distributed **around** each item.
- `space-between` → all available space placed **between** items.
- Add gaps between flex items with plain old `margin-left` / `margin-right`.

⚡ WHY IT MATTERS

Flexbox is everywhere – Twitter's left menu, tweet options, the timeline, "Trends for you" and "Who to follow" are all flexbox. **Every front-end developer must be comfortable with it.**

► Three ways to centre – compared

Technique	How	Verdict
<code>text-align</code>	<code>text-align: center</code> on the <i>parent</i>	Simple but <i>inflexible</i>
auto margins	block + width + <code>margin: auto</code>	Good for a <i>single</i> block
flexbox	<code>display: flex</code> + <code>justify-content: center</code>	Harder but super flexible – the pro way

⚠ DON'T MIX TECHNIQUES

`text-align` works on **all inline elements**, not just text. But never apply two centering techniques at once – they affect each other and just cause confusion. Remove `display: block` + auto margins before switching to flexbox.

18 Worked example - Google search field ★★★★★

A real design spec, read straight off the box model in dev tools:

Spec	Value
Content height	24px
Padding top/bottom	10px
Padding left/right	30px
Border thickness	1px
Border radius	24px
Width	400px

```
.search-input {
  width: 400px;
  line-height: 24px;
  padding-top: 10px;
  padding-bottom: 10px;
  padding-left: 30px;
  padding-right: 30px;
  border: 1px solid #dfe1e5;
  border-radius: 24px;
}
```

⚠ HEIGHT vs LINE-HEIGHT on inputs

Some browsers honour a CSS **height** rule on an input, some don't. Stack Overflow's advice is to use **line-height** instead. Since an input holds exactly **one line of text**, line-height achieves the same inner height, more reliably.

Using **height** is still fine — "it's more important that you use what you're comfortable with".

► Worked example — the Twitter "Following" button

```
button {
  color: #0f1419;
  background: white;
  border: 1px solid #cfd9de;
  border-radius: 20px;
  padding-top: 10px;
  padding-bottom: 10px;
  padding-left: 20px;
  padding-right: 20px;
  font-weight: bold;
}
```

METHOD

1. Pick the **low-hanging fruit** — values you're given.
2. Spot the **most apparent difference** next.
3. Nudge values until it matches.

As the box grows via padding, the **border-radius** must grow too to keep the same soft look.

⚡ ACCESSIBILITY — CONTRAST

Google's own button background **#F8F9FA** on white fails a contrast test — ratio **1.05** when it **should be 4.5 or more**. Low contrast harms visually-impaired users. Check with an online contrast checker; the course deliberately uses a darker grey instead.

DEBUGGING WITH DEV TOOLS

Right-click → Inspect → **Elements** tab → **Styles** tab → scroll to the box-model diagram. **Hover each layer** — margin lights up orange, padding green, content blue in the page. Great way to verify your numbers match your CSS. Chrome: the ☰ icon → "Dock side" to move the panel right.

⚡ QUICK REVISION

► Tag cheat-sheet

Tag	Purpose
<code><h1>...<h6></code>	Headings
<code><p></code>	Paragraph
<code></code>	Image ⚡ self-closing
<code><div></code>	Invisible container
<code><button></code>	Clickable button

Tag	Purpose
<code><input type></code>	Input ⚡ self-closing
<code><a href></code>	Link (anchor)
<code> </code>	Lists
<code><head> <body></code>	Metadata / visible
<code><link rel href></code>	Attach CSS file

► Must-know CSS recipes

CENTRE ONE BLOCK

```
display: block;
width: 300px;
margin-left: auto;
margin-right: auto;
```

CENTRE A ROW (PRO)

```
display: flex;
justify-content: center;

/* gaps → margin on children */
```

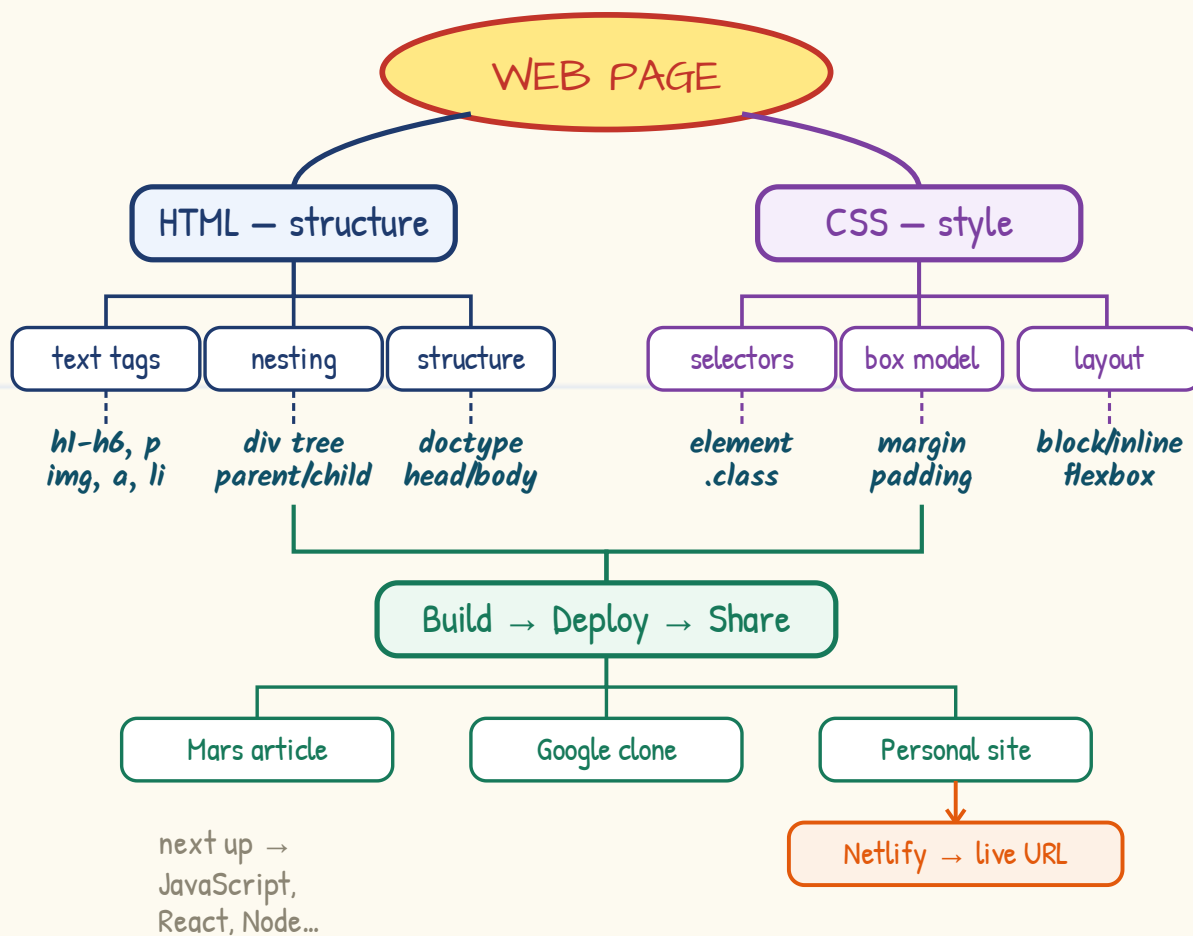
► 10 ultra-important exam facts

1. **HTML** = structure, **CSS** = style, **JavaScript** = behaviour.
2. Only text **between opening & closing tags** is formatted.
3. `<img>` and `<input>` are **self-closing**; the closing slash is optional.
4. `src` = source, `href` = **hypertext reference**, `alt` = alternative text.
5. `target="_blank"` opens the link in a **new tab**.
6. **"Display block stacks on top"** – blocks claim all horizontal space.
7. Centering with margins needs **all three**: block + width + auto margins.
8. **Vertical margins collapse** – the larger value wins, they never add up.
9. `margin` = space outside, `padding` = space inside, border sits between.
10. `<!doctype html>` declares **HTML5** – it is not a tag.

► Common mistakes & confusion traps

✗ Mistake	✓ Correct
<code>margin:auto</code> on a button does nothing	Buttons are inline – add <code>display:block</code>
Styling <code>div { }</code> directly	Use a class – element selectors hit every div
Expecting 15px + 20px = 35px gap	Collapsing margins → 20px
Forgetting the <code>;</code> or <code>!</code>	Breaks the whole rule
<code></code> as the real fix	Valid HTML uses CSS for width
Omitting <code>alt</code>	Hurts accessibility and SEO ranking

MIND MAP - CHAPTER AT A GLANCE



⚡ FINAL WORD

You now know the core HTML tags, nesting, document structure, lists, links, CSS syntax, the box model, three centering techniques and flexbox – enough to build and deploy a real website. *Take a break, digest, then move on to JavaScript.*